

1. MATLAB AND MATRIX MANIPULATION

AIM:

To study the fundamentals of image processing using MATLAB and the Image Processing Toolbox software.

INTRODUCTION:

MATLAB (short for Matrix Laboratory) is a high-level language and interactive environment developed by MathWorks. It is widely used for numerical computation, data analysis, algorithm development, and visualization. MATLAB excels in matrix and linear algebra operations, making it a go-to tool for engineers and scientists. It supports integration with Simulink, which enables system-level simulation and model-based design. MATLAB is powerful for prototyping, automation, and hardware interfacing with minimal lines of code.

FUNDAMENTALS OF DIGITAL IMAGES:

- A digital image consists of pixels arranged in rows and columns.
- Image dimensions are typically in powers of 2 (e.g., 512×512 pixels).
- The pixel values represent intensity (grayscale) or color information.

IMAGE FORMATS SUPPORTED BY MATLAB:

Format Name	Description	Extension
BMP	Windows Bitmap	.bmp
HDF	Hierarchical Data Format	.hdf
JPEG	Joint Photographic Experts Group	.jpg
PCX	Paintbrush Bitmap Image	.pcx
TIFF	Tagged Image File Format	.tif
XWD	X Window Dump	.xwd

IMAGE TYPES IN MATLAB:

1. **Intensity (Grayscale) Image**
 - double: Pixel values $\in [0, 1]$
 - uint8: Pixel values $\in [0, 255]$
2. **Binary Image**
 - Pixel values: 0 (black), 1 (white)

3. Indexed Image

- Image matrix + color map matrix

4. RGB Image

- Three matrices for Red, Green, Blue channels

5. Multiframe Image

- Sequence of images (e.g., time-series or CT scan slices)

MAJOR COMPONENTS OF MATLAB ENVIRONMENT:

- Command Window – Interface to enter commands and execute code.
- Workspace – Shows all active variables and their current values.
- Editor/Debugger – Used to write, save, and debug scripts and functions.
- Command History – Logs all previously executed commands.
- Current Folder Browser – View and manage files in the working directory.
- Figure Window – Displays graphical output like plots and charts.
- Simulink Library Browser – Access models and blocks for system simulations.
- Help Browser – Integrated documentation and support search engine.

COMMON FORMAT CONVERSIONS IN MATLAB:

Operation	MATLAB Command
Convert Intensity/Indexed/RGB to Binary	dither()
Convert Intensity to Indexed	gray2ind()
Convert Indexed to Intensity	ind2gray()
Convert Indexed to RGB	ind2rgb()
Convert Matrix to Intensity	mat2gray()
Convert RGB to Intensity	rgb2gray()
Convert RGB to Indexed	rgb2ind()

INDEXING AND REFERENCING ELEMENTS IN MATLAB:

Indexing is a fundamental aspect of array and matrix manipulation in MATLAB. It allows users to access, reference, and modify specific elements or subsets of data structures. MATLAB utilizes **1-based indexing**, meaning that the indexing of arrays begins from 1, not 0 as in many other programming languages. Understanding different types of indexing is crucial for performing data analysis, implementing algorithms, and optimizing code performance.

1. Scalar Indexing (Accessing a Single Element)

Scalar indexing is the most basic form of indexing, used to access individual elements within an array or matrix. The syntax involves specifying the row and column indices within parentheses.

Syntax: element = A(row, column);

Example: Consider a 3×3 matrix:

```
A = [10 20 30; 40 50 60; 70 80 90];
```

To access the element located at the second row and third column:

```
x = A(2,3); % Result: x = 60
```

2. Vector Indexing (Accessing Rows or Columns)

Vector indexing enables the extraction of an entire row or column from a matrix. This is particularly useful for operations involving slices of data.

- **Accessing an entire row:**

```
rowVector = A(rowIndex, :);
```

- **Accessing an entire column:**

```
columnVector = A(:, columnIndex);
```

Example: `row2 = A(2, :);` % Returns: [40 50 60]

`col3 = A(:, 3);` % Returns: [30; 60; 90]

3. Linear Indexing

In MATLAB, matrices are stored in **column-major order**, which means elements are arranged column-wise in memory. Linear indexing allows referencing elements using a single index, regardless of matrix dimensions.

Example: `A = [10 20 30; 40 50 60; 70 80 90];`

`element = A(5);` % Refers to the 5th element in column-major order → element = 60

The order of storage is:

```
[10; 40; 70; 20; 50; 80; 30; 60; 90]
```

4. Logical Indexing

Logical indexing provides a powerful way to access elements that satisfy specific conditions. A logical array (mask) is created by applying a relational operator to the array, and this mask is then used to extract elements that meet the condition.

Example: `A = [10 25; 5 40];`

```
logicalMask = A > 20; % Returns: [false true; false true]
```

```
selectedElements = A(logicalMask); % Returns: [25; 40]
```

Logical indexing is highly efficient in data filtering and conditional data extraction.

5. Indexing with the end Keyword

MATLAB provides the special keyword `end` to reference the last element in a dimension, which is especially useful when working with dynamically-sized arrays.

Examples:

```
lastElement = A(end); % Returns the last element in column-major order
```

```
lastRow = A(end, :); % Returns the last row of matrix A
```

```
lastColumn = A(:, end); % Returns the last column of matrix A
```

The use of end simplifies operations on matrices without requiring knowledge of their exact size.

6. Modifying Elements Using Indexing

Indexing is not only used for reading values but also for assigning or modifying specific elements or groups of elements in a matrix.

Example:

```
A(1,2) = 99;
```

This command modifies the element at the first row and second column of matrix A, changing it to 99.

DISPLAY IMAGES AND IMAGE SEQUENCES:

Function	Description
imshow	Display image in 2-D viewer
imblend	Blend two images
imfuse	Create composite of two images
immovie	Make movie from multiframe image
imshow	Display image
imshowpair	Compare differences between two images
montage	Display multiple images as a rectangular montage
viewer2d	Create viewer for 2-D image display
warp	Display image as a texture-mapped surface

DATA TYPE CONVERSION:

- **For processing:** Use double (high precision).
- **For storage:** Use uint8 (less memory).
- Conversion between types:

```
doubleImage = im2double(uint8Image); % uint8 to double
```

```
uint8Image = im2uint8(doubleImage); % double to uint8
```

CONCLUSION:

This experiment demonstrated basic image operations in MATLAB, including loading, displaying, and converting images between various formats and data types, laying the foundation for more advanced image processing tasks.